

Aula 7 (Módulo 03) — HTTP/HTTPS, TLS, Cookies, Sessões e APIs

Esta é a aula de consolidação do Módulo 03. Tudo que vimos até aqui (IP, porta, TCP, DNS) existe para, no fim, entregar o protocolo que domina o tráfego da internet moderna: **HTTP**.

1. HTTP — o protocolo da web

HTTP (*HyperText Transfer Protocol*) é como o navegador (cliente) conversa com um servidor web — pedidos (**requests**) e respostas (**responses**), seguindo o modelo cliente/servidor que já vimos no Módulo 00, Aula 3.

Métodos HTTP mais comuns

Método	Uso
GET	pedir um recurso (ex.: carregar uma página)
POST	enviar dados (ex.: submeter um formulário de login)
PUT	atualizar um recurso existente
DELETE	remover um recurso

Códigos de status HTTP

Faixa	Significado
2xx	sucesso (ex.: 200 OK)
3xx	redirecionamento (ex.: 301 Moved Permanently)
4xx	erro do cliente (ex.: 401 Unauthorized , 403 Forbidden , 404 Not Found)
5xx	erro do servidor (ex.: 500 Internal Server Error)

Por que importa para segurança: em logs de proxy/servidor web (Aula 6), esses códigos contam uma história. Muitos 401 / 403 seguidos do mesmo IP para caminhos diferentes de um site pode indicar tentativa de acesso não autorizado ou reconhecimento de aplicação; muitos 404 seguidos, testando caminhos variados (/admin , /backup , /.env), é um padrão clássico de varredura (scanning) em busca de arquivos/rotas sensíveis esquecidos expostos.

2. HTTPS e TLS — HTTP com criptografia

HTTPS é HTTP rodando **dentro** de uma camada de criptografia chamada **TLS** (*Transport Layer Security* — sucessor do antigo SSL, termo que ainda aparece por herança histórica, mas SSL em si está obsoleto). TLS garante três coisas:

- **Confidencialidade:** ninguém no meio do caminho consegue ler o conteúdo.
- **Integridade:** ninguém consegue alterar os dados sem ser detectado.

- **Autenticidade:** você tem uma garantia razoável de estar falando com o servidor real (não um impostor) — isso vem do **certificado digital**.

Certificado digital — como a autenticidade é garantida

Um certificado é emitido por uma **Autoridade Certificadora (CA)** confiável (ex.: Let's Encrypt, DigiCert), atestando "eu confirmei que este domínio pertence a esta organização/chave pública". Seu navegador já vem com uma lista de CAs em quem confia por padrão; se um site apresenta um certificado válido, assinado por uma dessas CAs, o cadeado aparece na barra de endereço.

Por que importa para segurança: um certificado **inválido, expirado ou autoassinado** (sem uma CA confiável por trás) gera um alerta forte no navegador — e é exatamente esse tipo de aviso que phishing sofisticado tenta contornar (registrando certificados válidos, mas para domínios de typosquatting, como vimos na Aula 4). Certificado válido garante que você está falando com o dono **daquele domínio específico** — não garante que o domínio em si é confiável (`paypal-login.com` pode perfeitamente ter um certificado HTTPS válido e legítimo, emitido para ele mesmo).

3. Cookies e sessões

HTTP é, por natureza, **stateless** (sem estado) — cada request é independente, o servidor não "lembra" de você entre um pedido e outro por padrão. **Cookies** resolvem isso: um pequeno pedaço de dado que o servidor pede para o navegador guardar e reenviar em cada request seguinte.

O uso mais importante de cookie é manter uma **sessão**: depois que você faz login, o servidor gera um **identificador de sessão** único e envia como cookie; o navegador reenvia esse cookie em cada request seguinte, e o servidor usa isso para saber "ah, esse é o usuário que já fez login".

Por que importa para segurança: roubar o cookie de sessão de alguém (via um ataque como **XSS**, que veremos no Módulo 18) permite ao atacante **se passar por aquele usuário** sem precisar da senha — chamado de **session hijacking**. Por isso, cookies de sessão devem ter atributos de proteção: `Secure` (só enviado via HTTPS), `HttpOnly` (inacessível via JavaScript, dificultando roubo via XSS), e `SameSite` (limita em quais contextos o cookie é enviado, mitigando ataques como CSRF).

4. APIs — a web conversando com a web

API (*Application Programming Interface*), no contexto web, geralmente se refere a um servidor que responde não com páginas HTML para humanos, mas com **dados estruturados** (frequentemente em **JSON**) para outros programas consumirem. O padrão mais comum é **REST**, que usa os mesmos métodos HTTP (`GET` / `POST` / `PUT` / `DELETE`) de forma padronizada sobre "recursos" (ex.: `GET /usuarios/123`).

Autenticação em APIs geralmente não usa cookies de sessão tradicionais, e sim **tokens** (como **API keys** ou **JWT — JSON Web Token**) enviados em um cabeçalho da requisição (`Authorization: Bearer <token>`).

Por que importa para segurança: APIs mal protegidas (sem autenticação adequada, ou com autorização mal implementada — permitindo que um usuário autenticado acesse dados de **outro** usuário só trocando um ID na URL, um problema chamado **IDOR**) são uma das categorias de vulnerabilidade mais comuns em aplicações modernas. Aprofundaremos isso — junto com XSS, CSRF e IDOR — no Módulo 18 (Web Security).

5. Conectando tudo — o Módulo 03 inteiro

```
DNS resolve nome → IP (Aula 4)
  → TCP three-way handshake na porta 443 (Aula 3)
    → TLS handshake: troca de certificado, negociação de chave (esta aula)
      → HTTP request dentro do túnel criptografado
        → Cookie de sessão identifica o usuário logado
          → Servidor pode ser uma API respondendo JSON
```

Essa cadeia completa — do nome de domínio até a resposta da aplicação — é o que você vai revisitar constantemente ao analisar tráfego web em Network Security (Módulo 04), Web Security (Módulo 18) e em praticamente toda investigação que envolva navegação/aplicações web.

6. Exercício prático

Abra as **Ferramentas de Desenvolvedor** do seu navegador (F12 → aba "Network"/"Rede") e visite qualquer site. Observe:

1. O código de status de cada requisição (a maioria deve ser `200`).
2. O cadeado ao lado do endereço — clique nele para ver detalhes do certificado (emissor/CA, validade).
3. Na aba "Application"/"Aplicativo" (Chrome) ou "Storage" (Firefox), veja os cookies daquele site e observe se têm as flags `Secure` e `HttpOnly` marcadas.

7. Recapitulando o Módulo 03

- HTTP = protocolo da web (métodos, códigos de status); padrões de `401` / `403` / `404` em massa indicam ataque/reconhecimento.
- HTTPS = HTTP + TLS (confidencialidade, integridade, autenticidade via certificado); certificado válido garante identidade do domínio, não que o domínio seja confiável.
- Cookies mantêm sessão sobre um protocolo sem estado; roubo de cookie de sessão = session hijacking; `Secure` / `HttpOnly` / `SameSite` mitigam isso.
- APIs (geralmente REST + JSON) usam tokens em vez de cookies; falhas de autorização (IDOR) são um risco central.

Com os sete módulos deste bloco de rede, você já tem o vocabulário completo do caminho de um pacote: do cabo Ethernet (MAC) até a resposta JSON de uma API, passando por IP, TCP/UDP, DNS, firewall/proxy/VPN e TLS.

Módulo 03 concluído. 🎉

Próximo módulo

Módulo 04 — Network Security: agora que a teoria está sólida, vamos usar ferramentas de verdade para observar e analisar tráfego — Wireshark, Zeek, Suricata, e a prática de análise de PCAP.

Fontes recomendadas

- **RFC 9110 — "HTTP Semantics"**: especificação técnica oficial (IETF) atualizada do HTTP. <https://www.rfc-editor.org/rfc/rfc9110>
- **RFC 8446 — "The Transport Layer Security (TLS) Protocol Version 1.3"**: especificação técnica oficial do TLS 1.3 (versão atual recomendada). <https://www.rfc-editor.org/rfc/rfc8446>
- **OWASP — "Session Management Cheat Sheet"**: boas práticas oficiais da OWASP para cookies de sessão seguros (`Secure` , `HttpOnly` , `SameSite`).
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- **OWASP — "API Security Top 10"**: referência oficial dos riscos mais comuns em APIs, incluindo IDOR (Broken Object Level Authorization). <https://owasp.org/API-Security/editions/2023/en/0x00-header/>